

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: DESIGN AND ANALYSIS OF ALGORITHMS

COURSE CODE: CSA06

NAME:K.NARENDRA REDDY

REG NO:192311236

ANNEXURE A

Q14. Distributed Sensor Network Analysis

Given:

$$T(n) = 3T\left(\frac{n}{3}\right) + 1$$

Step 1: Expand the recurrence

$$\begin{aligned}T(n) &= 3\left[T\left(\frac{n}{3}\right) + 1\right] + 1 \\&= 3^2T\left(\frac{n}{3^2}\right) + 3 + 1\end{aligned}$$

Expanding again:

$$= 3^3T\left(\frac{n}{3^3}\right) + 3^2 + 3 + 1$$

After k expansions:

$$T(n) = 3^kT\left(\frac{n}{3^k}\right) + \sum_{i=0}^{k-1} 3^i$$

Step 2: Find recursion depth

The recursion stops when:

$$\begin{aligned}\frac{n}{3^k} &= 1 \\ 3^k &= n \\ k &= \log_3 n\end{aligned}$$

Step 3: Substitute $k = \log_3 n$

$$T(n) = 3^{\log_3 n} T(1) + \sum_{i=0}^{\log_3 n - 1} 3^i$$

Since:

$$3^{\log_3 n} = n$$

and

$$\sum_{i=0}^{m-1} 3^i = \frac{3^m - 1}{2}$$

Therefore,

$$T(n) = nT(1) + \frac{n-1}{2}$$

Assuming $T(1) = c$,

$$\begin{aligned}T(n) &= cn + \frac{n-1}{2} \\ T(n) &= \Theta(n)\end{aligned}$$

Final Complexity

$$\boxed{T(n) = \Theta(n)}$$

Scalability Analysis

- Recursion depth grows as $\log_3 n$.
- Number of subproblems grows by a factor of 3 at each level.
- Total work across all levels is proportional to n .

Interpretation: As sensor nodes increase, execution time increases linearly, making the algorithm scalable for large distributed sensor networks.

Q15. Multi-Source Log Processing System

Given:

$$T(n) = 2T\left(\frac{n}{2}\right) + \log n$$

Step 1: Expand recurrence

$$\begin{aligned} T(n) &= 2 \left[2T\left(\frac{n}{4}\right) + \log\left(\frac{n}{2}\right) \right] + \log n \\ &= 2^2 T\left(\frac{n}{4}\right) + 2 \log\left(\frac{n}{2}\right) + \log n \end{aligned}$$

Expand again:

$$= 2^3 T\left(\frac{n}{8}\right) + 4 \log\left(\frac{n}{4}\right) + 2 \log\left(\frac{n}{2}\right) + \log n$$

After k levels:

$$T(n) = 2^k T\left(\frac{n}{2^k}\right) + \sum_{i=0}^{k-1} 2^i \log\left(\frac{n}{2^i}\right)$$

Step 2: Determine recursion depth

$$\begin{aligned} \frac{n}{2^k} &= 1 \\ k &= \log_2 n \end{aligned}$$

Step 3: Substitute

$$T(n) = nT(1) + \sum_{i=0}^{\log n - 1} 2^i (\log n - i)$$

Split the summation:

$$= (\log n) \sum 2^i - \sum i 2^i$$

Known results:

$$\sum_{i=0}^{m-1} 2^i = 2^m - 1 = \Theta(n)$$

and

$$\sum_{i=0}^{m-1} i 2^i = \Theta(n \log n)$$

Hence the additional work is:

$$\Theta(n)$$

Therefore,

$$T(n) = \Theta(n) + \Theta(n)$$
$$\boxed{T(n) = \Theta(n)}$$

Performance Analysis

- Recursive splitting creates $\log n$ levels.
- Merging cost is logarithmic at each node.
- Total accumulated cost remains linear.

Final Complexity

$$\boxed{\Theta(n)}$$

Interpretation: The system scales efficiently for very large log datasets; processing time grows proportionally to the number of log entries.

Q17. Hierarchical Tree Query Engine

Given:

$$T(n) = T\left(\frac{n}{2}\right) + \log n$$

Step 1: Expand

$$\begin{aligned} T(n) &= T\left(\frac{n}{4}\right) + \log\left(\frac{n}{2}\right) + \log n \\ &= T\left(\frac{n}{8}\right) + \log\left(\frac{n}{4}\right) + \log\left(\frac{n}{2}\right) + \log n \end{aligned}$$

After k expansions:

$$T(n) = T\left(\frac{n}{2^k}\right) + \sum_{i=0}^{k-1} \log\left(\frac{n}{2^i}\right)$$

Step 2: Recursion depth

$$\begin{aligned}\frac{n}{2^k} &= 1 \\ k &= \log_2 n\end{aligned}$$

Step 3: Substitute

Let $m = \log_2 n$.

$$\begin{aligned}T(n) &= T(1) + \sum_{i=0}^{m-1} (m - i) \\ &= T(1) + m + (m - 1) + \cdots + 1 \\ &= T(1) + \frac{m(m + 1)}{2}\end{aligned}$$

Substituting $m = \log n$:

$$T(n) = \Theta((\log n)^2)$$

Final Complexity

$$\boxed{T(n) = \Theta((\log n)^2)}$$

Scalability Analysis

- Depth of recursion is $\log n$.
- Work decreases gradually through levels.
- Total cost grows as the square of the logarithm.

Interpretation: Extremely scalable; doubling the data causes only a small increase in running time.

Q18. Intelligent Search Optimization System

Given:

$$T(n) = T\left(\frac{n}{2}\right) + \log n$$

This recurrence is identical to Q17.

Expansion

$$\begin{aligned}T(n) &= T\left(\frac{n}{4}\right) + \log\left(\frac{n}{2}\right) + \log n \\&= T\left(\frac{n}{8}\right) + \log\left(\frac{n}{4}\right) + \log\left(\frac{n}{2}\right) + \log n\end{aligned}$$

After k levels:

$$T(n) = T\left(\frac{n}{2^k}\right) + \sum_{i=0}^{k-1} \log\left(\frac{n}{2^i}\right)$$

Recursion Depth

$$k = \log_2 n$$

Let $m = \log_2 n$.

$$\begin{aligned}T(n) &= T(1) + \sum_{i=0}^{m-1} (m - i) \\&= T(1) + \frac{m(m+1)}{2} \\&= \Theta((\log n)^2)\end{aligned}$$

Final Complexity

$$\boxed{T(n) = \Theta((\log n)^2)}$$

Effect of Logarithmic Work

Each recursive level contributes a logarithmic amount of work, while the number of levels is also logarithmic.

Thus:

$$(\log n) \times (\log n) = (\log n)^2$$

Interpretation

The search optimization system remains highly efficient even for huge datasets because growth is only polylogarithmic.

Q19. Recursive Financial Summation Engine

Given:

$$T(n) = T\left(\frac{n}{2}\right) + n$$

Step 1: Expand recurrence

$$\begin{aligned} T(n) &= \left[T\left(\frac{n}{4}\right) + \frac{n}{2} \right] + n \\ &= T\left(\frac{n}{4}\right) + n + \frac{n}{2} \end{aligned}$$

Expand again:

$$= T\left(\frac{n}{8}\right) + n + \frac{n}{2} + \frac{n}{4}$$

After k expansions:

$$T(n) = T\left(\frac{n}{2^k}\right) + n \sum_{i=0}^{k-1} \left(\frac{1}{2}\right)^i$$

Step 2: Find recursion depth

$$\begin{aligned} \frac{n}{2^k} &= 1 \\ k &= \log_2 n \end{aligned}$$

Step 3: Substitute

$$T(n) = T(1) + n \sum_{i=0}^{\log n - 1} \left(\frac{1}{2}\right)^i$$

The geometric series:

$$\sum_{i=0}^{m-1} \left(\frac{1}{2}\right)^i = \frac{1 - (1/2)^m}{1 - 1/2} = 2 \left(1 - \frac{1}{2^m}\right)$$

Since $2^m = n$,

$$= 2 \left(1 - \frac{1}{n}\right)$$

Therefore,

$$T(n) = T(1) + 2n - 2$$

$$T(n) = \Theta(n)$$

Verification

The total work is:

$$n + \frac{n}{2} + \frac{n}{4} + \cdots + 1$$

which sums to less than $2n$.

Hence:

$$\boxed{T(n) = \Theta(n)}$$

Scalability Analysis

- Recursion depth is $\log n$.
- Work forms a decreasing geometric series.
- Total cost remains proportional to n .

Final Complexity

$$\boxed{\Theta(n)}$$

Interpretation: For large-scale transaction datasets, execution time grows linearly with the number of records, providing efficient and predictable performance.